

VERIQO Coverage, Validity and Self-Doubt

Round: H **Date:** 2026-09-04 **Branch:** `claude/l99-gap-coverage-nv70zy` **Input:** `Masalah_terbesar_Article_18_justru_8_REFUSED_dan_lainnya.docx`

0. The finding this round is built on

The review's opening sentence is the whole round:

Masalah terbesar Article 18 justru **8 REFUSED**.

Eight of the twenty-three corpus variants were refused rather than processed, and the report that shipped them said "15 of 23, and every refusal is safe". Both halves of that sentence were true. Together they were misleading, because two of the eight refusals were `/ObjStm` (object streams) and `/XRef` (cross-reference streams) — and those are not exotic structures. They are what **every PDF produced since Acrobat 6 (2003)** uses by default. Refusing them meant refusing most real documents while reporting 65% coverage.

The review's decomposition is the one this round adopts:

Dimension	Before this round
Safety	strong — nothing leaks, every refusal is fail-closed
Coverage	incomplete — and the incompleteness was in the common case
Qualification	no — nobody outside VERIQO has tested any of it

Three separate statements. The old report collapsed them into one number. This round separates them permanently, in code.

1. Object streams and cross-reference streams are now handled

`pkg/evidence/redaction/worker/objstm.go` (new).

The design decision worth stating: the worker **normalizes, then redacts**. It does not attempt to rewrite content inside a compressed container. It:

1. walks the document object by object, matching complete `N G obj ... endobj` spans rather than a single regex across the file (an earlier attempt did the latter and matched from one object's dictionary to a *later* object's stream);
2. for each `/Type /ObjStm`, inflates the stream, reads the `/N` pair table and `/First` offset, and lifts every contained object out to the top level as an ordinary indirect object;
3. drops `/Type /XRef` streams, recovering `/Root` and `/Info` from the xref stream's own dictionary;
4. rebuilds a classic cross-reference table and trailer.

There is a security argument for that order, not only a convenience one. An object stream that is never opened holds content **nobody inspected**. Lifting the objects means the redactor sees them. A pipeline that refuses the container is safe; a pipeline that opens it and verifies what was inside is safe *and* has looked.

`TransformManifest.Normalization` records what was done, so a reader of the release chain can see that the derivative's structure was changed before redaction and why.

The fixture was fake, and that is the review's own point

The first version of the `PDF-OBJECT-STREAM` and `PDF-XREF-STREAM` corpus fixtures did **not** build real containers. They wrote the forbidden term into an ordinary catalog dictionary and named the structure in a comment. Every test passed.

That is precisely the overfitting the review warned about, occurring inside our own corpus, in the same round we were told to watch for it. `pkg/evidence/redaction/corpus/pdf15.go` now builds a genuine `/ObjStm` with a real pair table and a real `/W [1 4 2]` binary cross-reference stream, and `TestTheObjectStreamFixtureIsGenuine` asserts that the forbidden term is **not findable in the fixture's raw bytes** until the container is unpacked. If somebody downgrades the fixture again, that test fails.

This is recorded as **FC-005** in the failure-class register (§5).

2. Real-World Weighted Coverage

`corpus.Coverage.WeightedCoverage()`.

Structural coverage counts variants. It treats a rare structure and a ubiquitous one as worth the same, which is the wrong question for a commercial user. Each variant now carries a `RealWorldWeight`:

Weight	Prevalence	Meaning
UBIQUITOUS	0.65	present in most documents of that format
COMMON	0.20	present in a substantial minority
OCCASIONAL	0.10	encountered regularly, not typical
RARE	0.02	encountered, but not in ordinary business documents

Current numbers, produced by `corpus.Run()` (headline abridged; the full text carries the estimate caveat in §2 below):

```
23 structural variants: 17 accepted, 6 refused by design, 0 failed.
Structural coverage 74% -- the share of VARIANTS the workers can redact,
NOT a pass rate.
Real-world weighted coverage 88% (ESTIMATE).
3 refused variant(s) are COMMON or UBIQUITOUS in real documents:
PDF-ENCRYPTED (COMMON), PDF-INCREMENTAL (COMMON), PPTX-EMBEDDED-IMAGE (COMMON).
```

Two honesty constraints are enforced rather than promised:

- `TestWeightedCoverageIsReportedAsAnEstimate` requires the word **ESTIMATE** to travel with the number. The prevalence weights are **VERIQO's stated estimates, not measurements** — VERIQO has never run this pipeline over a real document population. They are published so a reader can disagree with the estimate rather than with a hidden assumption, and so the figure can be recomputed when a real corpus provides better numbers.
- `TestTheCoverageRatioIsReportedAsCoverageNotAsAPassRate` and `TestTheWeightedGapIsReported` keep the two numbers from merging back into one.

88% is not an achievement claim. It is a statement that the remaining 12% is concentrated in three named structures, so anyone deciding whether to send VERIQO their documents knows what to test first.

3. VALIDITY: the second temporal dimension

pkg/provenance/temporal/validity.go (new).

Round G gave every reference a temporal **state** (CURRENT, HISTORICAL, SUPERSEDED, DERIVED, EXTERNAL, UNVERIFIED). The review pointed out that state alone cannot answer the question a user actually asks. A certificate that expired last month is **CURRENT** as a record and useless as authority.

So validity is a separate axis with six values:

Validity	Means
VALID	usable now
EXPIRED	was valid, is not now
VALID_AT_TIME	valid for the period it covers, not for now
INVALID_FOR_CURRENT	never applied to the present question
REVOKED	withdrawn by its issuer; supports nothing, at any time
UNASSESSED	the zero value — nobody has looked

Standing binds a Reference to a Validity over an interval and answers two different questions with two different methods: UsableNow() and UsableForItsPeriod(). Old evidence remains usable *for its period* — deleting that distinction is how systems end up discarding sound historical evidence, and how they end up citing expired authority as current.

Two combinations are refused outright: SUPERSEDED + VALID, and HISTORICAL + VALID. A superseded reference that still claims present validity is a contradiction, not a state.

4. Evidence Quality: nine attributes, no score

pkg/evidence/quality/quality.go (new).

Attribute	The question it asks
Authenticity	is it what it claims to be?
Integrity	is it unaltered since acquisition?
Provenance	is its origin traceable?
Completeness	is anything missing?
Independence	does it come from a party with no stake?
Freshness	is it current for the question?
Reproducibility	can it be obtained again?
Scope	does it cover the claim, or only part?
Authority	is the source entitled to say this?

Three design rules, each enforced by a test:

1. **There is no score.** A strong INTEGRITY does not offset an absent INDEPENDENCE, because those do not trade against each other. (TestStrongIntegrityDoesNotOffsetAbsentIndependence.)

2. **All nine are materialised**, defaulting to `NOT_ASSESSED`. Omission and non-assessment look identical in a map, and only one of them is a determination. (`TestAllNineAttributesAreMaterialised.`)
3. **UNASSESSABLE is checked before INSUFFICIENT**. A vector with unasked questions cannot be called insufficient — that would be a conclusion drawn from an absence of work. (`TestUnassessedIsNotInsufficient.`)

`ADEQUATE` requires a statement of what it does not cover, because `ADEQUATE` without limits is read as `STRONG` by every reader who is in a hurry.

It is wired into the qualification ladder, not parked beside it

This is the part that makes it more than a vocabulary. `ledger.Entry` now carries an optional `*quality.Assessment`, and `Entry.Validate()` enforces four joins:

- a **PASS** may not rest on evidence assessed `INSUFFICIENT` (`ErrEvidenceInsufficient`) — though the same evidence may still be recorded as a `FAIL`, because pushing the deficiency out of the ledger would be the opposite of what the ledger is for;
- a level that **requires an outside party** (`QUALIFIED` and above) requires an assessment to exist (`ErrEvidenceUnassessed`) — that is the step where somebody outside is supposed to have looked at the *evidence*, not at the claim;
- an **incomplete** assessment may accompany any result up to the internal ceiling and nothing above it;
- the limits an `ADEQUATE` grade states must appear in the entry's own `Limitations` (`ErrLimitsDropped`) — limits travel with the conclusion, or the conclusion is being presented as unbounded.

The assessment is covered by the entry hash (`TestTheAssessmentIsCoveredByTheEntryHash`), so it cannot be edited after the entry is appended without breaking the chain. Without that, the assessment would be a comment.

5. Failure-class discipline

`pkg/assurance/failureclass/` (new).

Every finding VERIQO has closed so far was closed by fixing the site that was found. That is necessary and not sufficient: the site is one instance, and the **class** is what recurs. The discipline is eight stages, and a response that stops early is a patch:

```
FINDING -> ROOT CAUSE -> FAILURE CLASS -> INVARIANT
        -> POSITIVE TEST -> NEGATIVE TEST -> MUTATION TEST -> REGRESSION TEST
```

The last three are the ones that get skipped:

- a **negative** test proves the control refuses what it was designed to refuse — the control working, not the claim holding;
- a **mutation** test constructs the forbidden state by hand, around the normal path, and demands rejection anyway;
- a **regression** test governs the whole module rather than the one site.

Ten real findings from rounds CS, F, G and H are carried through all eight stages, across ten named failure classes:

ID	Class	Finding
FC-001	AUTHORITY_DIFFUSION	<code>caseproofgraph</code> could mint Findings

ID	Class	Finding
FC-002	VACUOUS_VERIFICATION	byte search over a compressed container
FC-003	UNASSESSED_TREATED_AS_ASSESSED	cluster count used as corroboration count
FC-004	STALE_CITATION	nine articles cited tests that no longer existed
FC-005	FIXTURE_NOT_GENUINE	the PDF 1.5 fixtures were not real containers
FC-006	SELF_QUALIFICATION	VERIQO could be its own external validator
FC-007	SEQUENCING_BYPASS	resolve first, prove later
FC-008	SCOPE_OVERCLAIM	coverage ratio read as a pass rate
FC-009	OFFSETTING_ATTRIBUTES	strong integrity standing in for absent independence
FC-010	IRREVERSIBILITY_OVERCLAIM	visual-only redaction called irreversible

`Entry.Validate()` refuses a chain that stops early, refuses two stages citing the same test, and refuses an "invariant" phrased as a description rather than as a rule that can be violated.

`TestEveryFailureClassCitationNamesATestThatExists` resolves all forty cited test names against the module. That is FC-004 applied to the register that records FC-004 — which is the point: a class is closed when the invariant governs everything of that shape, **including the thing that recorded it**.

6. Mutation qualification

`test/mutation/` (new). Five mutations, each constructing the forbidden value directly:

Mutation	Must be rejected because
<code>UNKNOWN</code> → <code>INDEPENDENT</code>	Article 28
<code>REFUSED</code> → <code>FAILED</code>	a refusal is safe behaviour, not a capability result
<code>HISTORICAL</code> → <code>CURRENT</code>	promotion needs a reason; demotion does not
<code>NOT_ASSESSED</code> → <code>ASSESSED</code>	an absence of work is not a determination
<code>SELF</code> → <code>EXTERNAL</code> validator	VERIQO cannot validate VERIQO

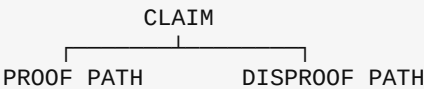
All five are rejected. `TestEveryMutationIsAttempted` prevents the suite from quietly shrinking.

The reason this matters more than the count: **a surviving mutant is a hole whether or not any test exercises it**. A suite that only ever runs the paths the code already takes cannot distinguish "the controls hold" from "the tests match the code".

7. The Self-Doubt Principle

`pkg/assurance/selfdoubt/` (new).

Setiap claim yang dibuat VERIQO harus mempunyai mekanisme untuk mencoba membuktikan claim tersebut **salah**.





Every test in this repository is a proof path. None of them is designed to make a claim FALSE. The register refuses a claim with no disproof path, refuses a disproof path that merely restates the proof path, and — the half most systems leave out — refuses a claim that recorded a counterexample and did not demote (`ErrDemotedButHeld`).

Eleven claims are registered. **Three are UNSETTLED, deliberately:**

Claim	Why it is not established
CLAIM-REDACTION-IRREVERSIBLE	no adversarial recovery lab has attempted reconstruction
CLAIM-REDACTION-REAL-WORLD-COVERAGE	the prevalence figures are estimates over a VERIQO-built corpus
CLAIM-FIVE-FABRICS-COMPOSE	never run on real rights-aware commercial data

A claim nobody tried to break is UNSETTLED, not established: it has been shown to *work*, which is a different thing from having been shown to *hold*.

The register's closing line is generated, not written:

All 8 established claim(s) were attacked by VERIQO alone; no outside party has tried to break any of them. Surviving one's own attack is the weakest form of survival available.

8. Article 18 after this round

	Before	After
Structural coverage	15/23 (65%)	17/23 (74%)
Real-world weighted	not measured	88% (ESTIMATE)
Object streams	REFUSED	handled (normalized)
Xref streams	REFUSED	handled (normalized)
Qualification	no	no

The verdict stays ASSURANCE_GAP and `Qualification: false`, **for a different reason than before** — which is worth stating, because a verdict that stays the same for a different reason is a verdict nobody re-examined. The old reason was that the workers refused the structures where redacted content most plausibly survives. Two of those are now handled. The remaining reason is the one that no amount of engineering closes:

No party outside VERIQO has attempted to recover redacted content from a derivative.

Six variants are still refused: encrypted documents, incremental updates, LZW-filtered streams, structurally malformed documents, PPTX embedded images, XLSX embedded binary objects. Three of those are COMMON in real documents. Refusing is safe. It is not the same as having proven those structures can be redacted.

Runtime record: `AUDIT-014-redaction.derivative_released` (`evidence/RUNTIME_EVIDENCE.json`, record 14, actor `compliance-1`).

9. Verification

`scripts/verify.sh` — **ALL CHECKS PASSED**, including:

- `go build ./...`, `go vet ./...`, `gofmt clean`;
- `full go test ./...` with coverage;
- `-race` repeat (5×) on the consensus-critical packages;
- `test/architecture` (36s), `test/adversarial`, `test/integration`, `test/mutation`, `test/soak` (121s), `test/stress`, `test/chaos`, `test/e2e/eight_blockers`.

`evidence/RUNTIME_EVIDENCE.json` regenerated: **byte-identical**. The chain this round touched is the assurance and qualification layer, not the evidence-to-decision chain, and the artefact says so by not changing.

Not run in this sandbox, unchanged from previous rounds and stated in `scripts/verify.sh` itself: `golangci-lint`, `govulncheck`, SPIRE/SPIFFE attestation, OPA bundle validation, `gosec` — all require network or external services this environment blocks.

10. What this round did not do

Stated plainly, because the round is about not overstating:

1. **Nothing here was validated externally.** Every new package was written, tested and attacked by VERIQO. The self-doubt register says so on every claim rather than once in a preface.
2. **The 88% is an estimate.** It becomes a measurement the first time the pipeline runs over documents VERIQO did not create (`VERIQO_CORPUS_DIR`, already wired, currently empty).
3. **Six variants are still refused**, three of them common.
4. **No control moved above ASSURED.** The internal ceiling is unchanged and unreachable from inside; that is the design, not a shortfall.

The one thing this round can claim is narrower and real: the gap between what VERIQO does and what VERIQO says it does is smaller than it was, and there are now four separate mechanisms — weighted coverage, the mutation suite, the failure-class register and the self-doubt register — whose only job is to keep it from widening again.